

PROJECT REPORT

AuraTune

An adaptive, explainable audio-EQ personalization engine
— perception → a 3-agent LangGraph pipeline → a
parametric DSP engine → your real EQ app's exact sliders.



Repository: github.com/Ansh-P1/auratune · Entry point: `app.py` (Streamlit) · Report generated 2026-09-15

Contents

- | | |
|-------------------------------|------------------------------|
| 1. What AuraTune does | 8. Streamlit dashboard |
| 2. Architecture & data flow | 9. What's real vs. simulated |
| 3. Perception layer | 10. Validation results |
| 4. Agent pipeline (LangGraph) | 11. Project structure |
| 5. DSP / parametric EQ engine | 12. Setup & running it |
| 6. Real-EQ projection | 13. Tech stack |
| 7. Data layer | |

1. What AuraTune does

AuraTune automatically re-tunes a listener's audio EQ in response to two things: **what's happening in the room** (ambient noise level) and **what's playing** (podcast, music, or movie). Instead of a static equalizer setting, it continuously reasons about the right curve and explains, in one plain-English sentence, why it changed.

Its standout feature: it doesn't just recommend an abstract curve — it can read a **photo of the user's actual EQ app** (Wavelet, Spotify, a car head unit, anything with sliders) and tell them the *exact* values to set on their *real* sliders, snapped to that app's step size and gain range.

Context-aware

Detects ambient noise level and content type, and adapts presence/bass automatically — noisier rooms get more vocal clarity and less muddying bass.

Command-driven

Accepts free-text requests like "make voices clearer" or "less bass, this room is boomy" and folds them into the curve.

Explainable

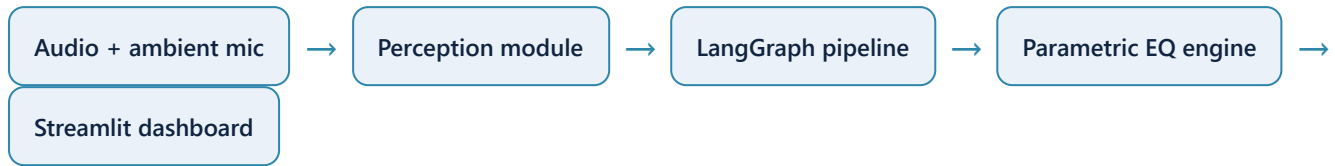
Every adjustment ships with a one-sentence, human explanation of what changed and why — generated by an LLM or a deterministic template.

Device-real

Projects the ideal curve onto the user's actual EQ app's sliders (frequencies, step size, gain range) instead of a generic recommendation.

2. Architecture & data flow

The system is a straight-line pipeline, matching the team's original architecture slide ([Grp_186__PPT.pptx](#)):



Concretely, one "Run adaptation" click in the UI does this:

1. **Perception** classifies the ambient noise level and content type from two audio buffers ([perception/context_classifier.py](#)).
2. **Profile Agent** loads the user's stored per-content-type target curve from the database ([agents/profile_agent.py](#)).
3. **EQ Decision Agent** blends that baseline with context-driven deltas (noise level) and any typed command, producing a new target curve ([agents/eq_decision_agent.py](#)).
4. **Projection Agent** snaps that continuous curve onto the user's real EQ app's slider grid, if one is set ([agents/projection_agent.py](#) → [dsp/eq_projection.py](#)).
5. **Explainer Agent** turns the numeric deltas into one plain-English sentence, plus a clause naming the exact slider moves ([agents/explainer_agent.py](#)).
6. The dashboard renders the before/after curve, the slider table, and the explanation, and the result is persisted back to the profile store as history.

Why LangGraph? The four agent steps are wired as a linear `StateGraph` ([agents/graph.py](#)) passing a single typed state dict through each node. This makes it trivial to insert a 5th node later (e.g. a Safety agent capping gain deltas) without touching existing agents.

3. Perception layer

Context classifier — `perception/context_classifier.py`

Takes an ambient-mic buffer and a content buffer, returns a `Context` (noise_level, content_type, ambient RMS, feature dict):

- **Noise level** — always signal-derived: RMS-in-dB thresholds split ambient audio into `quiet` (< -45 dB), `moderate` (-45 to -25 dB), and `noisy` (> -25 dB).
- **Content type** — a heuristic classifier (no trained model) using three signal features via `librosa`:
 - *Envelope modulation (2–8 Hz)* — speech's syllable rate gives a strong, narrow peak here; the main speech/non-speech discriminator.
 - *Spectral flatness* — separates tonal content (music) from noisier/percussive mixes.
 - *Harmonic/percussive energy ratio* — movies mix dense percussive/FX energy with dialogue and score.

A caller can also pass a `content_type_hint` (ground truth from a media player's own metadata, or a labeled demo scenario) to skip the heuristic entirely — the heuristic is a fallback, not the only path, mirroring how a real deployment would prefer an authoritative source when one exists.

EQ app reader — `perception/eq_app_reader.py`

Turns a screenshot of the user's real EQ app into an `EqualizerSpec` (band frequencies, gain range, step size) using a vision LLM. Tries **Google Gemini** first (free tier, REST call, no extra dependency), then **Anthropic Claude**; with neither key set it returns a friendly error and the UI falls back to manual entry — nothing crashes. Output is validated (1–32 bands, sane gain range, step clamped to a sensible value) before becoming a usable spec.

Stem separation — `perception/stem_separation.py`

Wraps **Demucs** (`htdemucs`) to split content into vocals / drums / bass / other stems. Demucs needs pretrained weights downloaded on first use; if that fails (offline/locked-down environment), it falls back to `librosa.effects.hpss` (harmonic/percussive split) — a coarser approximation that keeps the rest of the pipeline working.

Synthetic scenarios — `perception/synth_scenarios.py`

Generates representative audio standing in for real mic capture / media playback in the demo: a speech-like envelope for podcast, a stable harmonic stack for music, and dialogue+FX bursts for movie. This is the seam that gets swapped for real `sounddevice` / `streamlit-webrtc` capture in a live deployment — nothing downstream changes.

4. Agent pipeline (LangGraph)

Agent	File	Responsibility	Uses an LLM?
Profile	<code>profile_agent.py</code>	Reads the user's stored per-content-type target curve from the profile store; hands the graph a clean baseline <code>TargetCurve</code> .	No — deterministic data access
EQ Decision	<code>eq_decision_agent.py</code>	Blends the baseline with rule-based context deltas (noise level → presence/bass shift) and any typed command's parsed deltas; clamps every gain to a $\pm$ <code>MAX_GAIN_DB</code> ear-safety range.	Only for free-text command parsing — falls back to keyword-regex rules with no key
Projection	<code>projection_agent.py</code>	Snaps the decided curve onto the user's real EQ spec's slider grid (no-op if no spec is set).	No — pure interpolation/rounding
Explainer	<code>explainer_agent.py</code>	Turns the deltas into one plain-English sentence, plus a clause naming the exact slider moves for the user's app.	Yes — falls back to a templated sentence with no key

Context → curve rules (EQ Decision Agent)

Noise level	Presence delta	Bass delta	Rationale
quiet	+0.0 dB	+0.0 dB	No compensation needed
moderate	+1.5 dB	−1.0 dB	Light vocal boost, slight bass pullback
noisy	+3.5 dB	−2.5 dB	Strong vocal/presence boost; bass pulled back to avoid mud/stacking against ambient noise

Free-text commands are parsed into the same delta shape (`volume_db` , `bass_gain_db` , `presence_gain_db` , `treble_gain_db`) either by Claude (structured-JSON prompt) or, without an API key, by keyword-regex rules — e.g. "clearer voice / dialogue / speech" → +3 dB presence, "less bass / too boomy" → −3 dB bass, "brighter / more treble" → +2 dB treble, "louder / turn up" → +3 dB volume.

LLM client — `agents/llm_client.py`

A single thin wrapper (`complete(system, user)`) shared by the command parser and the Explainer. Returns `None` if `ANTHROPIC_API_KEY` isn't set or the call fails, so every caller has one clean fallback branch — the graph always produces a usable result, online or offline.

Graph wiring — `agents/graph.py`

```
profile_agent → eq_decision_agent → projection_agent → explainer_agent → END
```

State is a single `TypedDict` (`PipelineState`) threaded through every node. `run_pipeline()` is the one-shot convenience call used by both the Streamlit app and the validation script — after the graph finishes, it persists the decided curve back into the user's profile (so next time's baseline reflects it) and appends a history entry.

5. DSP / parametric EQ engine — `dsp/parametric_eq.py`

Pure signal processing, no ML, no network calls. A `TargetCurve` (volume, bass, presence, treble gains) expands into three biquad filters:

Band	Type	Centre frequency	Purpose
Bass	Low shelf	100 Hz	Overall low-end weight
Presence	Peaking	3 kHz	Vocal / dialogue clarity
Treble	High shelf	8 kHz	Air / brightness

Biquad coefficients are computed directly from the Audio-EQ-Cookbook formulas for each filter type.

`ParametricEQ` exposes:

- `apply(audio, curve)` — runs the filter chain via `scipy.signal.lfilter`, applies overall volume trim, and safety-clips output to $[-1, 1]$ to avoid blowing out speakers on large boosts.
- `frequency_response(curve)` — analytic magnitude response (via `scipy.signal.freqz`) across 20 Hz–Nyquist on a log scale, used both to draw the curve in the dashboard and to sample it when projecting onto real sliders.
- `delta(before, after)` — the numeric per-band deltas the Explainer Agent turns into a sentence.

6. Real-EQ projection — `dsp/equalizer_spec.py` & `dsp/eq_projection.py`

This is AuraTune's most distinctive piece of engineering. The agents reason about a smooth, continuous target curve — but every real EQ app is a *fixed grid*: N sliders at fixed frequencies, each with a limited gain range and a fixed step (some apps move in 1 dB, some in 0.5, Wavelet in 0.1 dB). `EqualizerSpec` captures that grid; `project_curve()` does the snapping:

1. Sample the ideal curve's frequency response at each of the app's actual slider frequencies (log-frequency interpolation).
2. Strip the curve's overall volume so only the *shape* is projected onto sliders.
3. Snap each sampled gain to the app's step size and clamp it to the app's min/max range, tracking which bands got clamped.
4. Compute a **preamp**: enough negative headroom to cancel the largest boost (so nothing clips), then fold in whatever overall level the curve asked for. Apps without a real preamp control still get a recommended number, shown as "lower the media/app volume by this much."
5. Report a **fit error** (RMS of set-vs-ideal gain across bands) so the user knows how closely their app's grid could match the ideal curve.

Built-in presets ship for **Wavelet** (9-band, 0.1 dB step — read from an actual screenshot), a classic **10-band ISO** graphic EQ, **Spotify's** 5-band EQ, and a 3-knob **car head unit**. Any spec — built-in, hand-typed, or read from a screenshot — can be saved into `eq_specs/*.json` and reused from the dropdown.

Example output sentence: "...On your Wavelet EQ, set 4 kHz +4.4, 62.5 Hz -4.3, 2 kHz +3.5... and drop the volume -4.4 dB."

7. Data layer — `data/db.py`

`ProfileStore` holds one document per user: an audiogram (reserved for a future hearing-sweep test), per-content-type target curves (podcast/music/movie baselines), and a running adjustment history. Backed by **MongoDB** when `MONGO_URI` is set and reachable (it pings on startup and fails fast); otherwise it transparently falls back to a local JSON file (`data/profiles.local.json`), so the whole pipeline runs offline with zero setup. The interface (`get_profile` / `save_profile` / `log_adjustment` / `get_history`) is identical either way — callers never know which backend is active.

8. Streamlit dashboard — `app.py`

The single entry point (`streamlit run app.py`). Left column: pick a simulated scenario (quiet+podcast / noisy+music / home+movie), optionally type a live command, and set up "Your EQ app" (screenshot upload, a built-in/saved preset, manual entry, or none) — plus a running history log. Right column, after clicking **Run adaptation**:

- **Detected context** — noise level, content type, ambient dB as metric tiles.
- **Live EQ curve** — a Plotly chart overlaying the stored baseline, the live adapted curve, and (if a spec is set) the actual slider points on the real EQ's grid.
- **Set these on <your EQ>** — a table of exact per-band values plus preamp, with a fit-quality caption and a downloadable `.txt` of the settings.
- **Explanation** — the Explainer Agent's one-sentence summary.
- **Raw deltas (debug)** — an expandable JSON dump of context/command deltas and the full projected EQ, for development/debugging.

9. What's real vs. simulated

Built in a sandboxed environment without live mic hardware, a running MongoDB instance, an API key, or network access to Meta's model hub — so every integration point is implemented for real and tested against a **graceful, documented fallback**, never stubbed out.

Component	Status	Detail
DSP engine (biquad filters, frequency response)	Real	Runs for real, no mocking.
Noise-level detection	Real	Fully signal-derived RMS thresholding; unit tested.
Content-type classification	Real heuristic	Real signal-derived heuristic, exercised directly in tests; demo scenarios pass a <code>content_type_hint</code> the way a real media player's metadata would, ahead of a trained classifier.
Ambient + content audio capture	Simulated	<code>synth_scenarios.py</code> generates representative audio in place of live mic/player capture — swap for <code>sounddevice</code> / <code>streamlit-webrtc</code> , nothing downstream changes.
Stem separation (Demucs)	Real + fallback	Real Demucs integration; automatic <code>librosa.hpss</code> fallback if pretrained weights can't be fetched.
Claude / LLM calls	Real + fallback	Real <code>anthropic</code> SDK calls when a key is set; deterministic rule-based fallback otherwise.
EQ-screenshot reading	Real + fallback	Real Gemini/Claude vision calls; falls back to manual entry with no key.
MongoDB profile store	Real + fallback	Real <code>pymongo</code> integration; transparent local-JSON fallback offline.
LangGraph pipeline wiring, EQ decision logic, Streamlit UI	Real	Runs for real, no mocking.

10. Validation results

Reproducible via `python3 validation/generate_traces.py` , output to `validation/output/` (one before/after curve PNG per scenario, plus `report.md`).

Scenario	Detected	Deltas (dB)	Explanation
Quiet room + podcast	quiet / podcast ambient −54.0 dB	all 0.0 — no change	"No change needed — your podcast curve already fits a quiet room."
Noisy environment + music	noisy / music ambient −21.9 dB	presence +3.5, bass −2.5	"Because of a noisy environment during music, I boosted vocal clarity by 3.5 dB, pulled bass back 2.5 dB."
Home + movie	moderate / movie ambient −36.5 dB	presence +1.5, bass −1.0	"Because of a moderate environment during movie, I boosted vocal clarity by 1.5 dB, pulled bass back 1.0 dB."

All three traces match the project's own validation plan expectations: minimal compensation in a quiet room, a vocal/presence boost with bass pulled back in a noisy environment, and correct curve switching purely on content-type change.

Unit tests

`tests/test_dsp.py` , `test_eq_projection.py` , `test_eq_app_reader.py` , and `test_classifier.py` (needs `librosa`) cover the biquad filter math, the curve-to-slider projection logic, screenshot-reading validation, and the noise/content classifiers respectively.

11. Project structure

```
auratune/
├─ app.py           Streamlit dashboard (entry point)
├─ config.py        env-driven settings
├─ dsp/
│   ├─ parametric_eq.py    biquad EQ engine + frequency response
│   ├─ equalizer_spec.py   describes a real EQ (bands, step, range) + presets
│   └─ eq_projection.py    snap the ideal curve onto a real EQ's sliders
├─ perception/
│   ├─ context_classifier.py noise level + content type
│   ├─ eq_app_reader.py     screenshot of an EQ app -> EqualizerSpec (vision)
│   ├─ stem_separation.py   Demucs wrapper (falls back to HPSS)
│   └─ synth_scenarios.py   synthetic audio for demos/validation
├─ agents/
│   ├─ profile_agent.py     reads/writes MongoDB profile
│   ├─ eq_decision_agent.py context + command -> target curve
│   ├─ projection_agent.py  target curve -> your EQ app's exact slider values
│   ├─ explainer_agent.py  deltas -> one plain-English sentence
│   ├─ llm_client.py        thin Anthropic API wrapper w/ fallback
│   └─ graph.py             LangGraph wiring of the agents
├─ data/
│   └─ db.py                MongoDB w/ local-JSON fallback
├─ eq_specs/               one JSON per real EQ app (e.g. from a screenshot)
├─ validation/
│   └─ generate_traces.py   generates the 3 required validation traces
└─ tests/
    ├─ test_dsp.py
    ├─ test_eq_projection.py
    ├─ test_eq_app_reader.py
    └─ test_classifier.py
```

12. Setup & running it

```
git clone https://github.com/Ansh-P1/auratune.git
cd auratune
python3 -m venv venv && source venv/bin/activate    # Windows: venv\Scripts\activate
pip install -r requirements.txt
streamlit run app.py
```

Runs with **zero external services** out of the box — no API key, no MongoDB, no network access to Meta's model hub all degrade to documented, working fallbacks (see section 9). To unlock the full pipeline:

```
export ANTHROPIC_API_KEY=sk-ant- ...    # free-text commands + natural explanations (+ screens
export GEMINI_API_KEY=AIza ...          # EQ-screenshot reading, free tier
export MONGO_URI=mongodb+srv:// ...    # persistent, shared profile storage
```

The app can also be deployed for free on [Streamlit Community Cloud](#) — point it at this repo, branch `main`, main file `app.py`.

13. Tech stack

DSP / perception numpy, scipy, librosa, soundfile	Stem separation torch, demucs (optional, HPSS fallback)
Agent pipeline langgraph, langchain-core, anthropic	Vision (screenshot reading) Google Gemini REST API, or Anthropic Claude
Data layer pymongo (optional, local-JSON fallback)	UI streamlit, plotly, matplotlib
Live capture (future) sounddevice, streamlit-webrtc	